

UNIVERSITY OF IOANNINA
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

ADVANCED TOPICS IN DATABASE TECHNOLOGY AND APPLICATIONS

PROGRAMMING ASSIGNMENT FOR
THE ACADEMIC YEAR 2025–2026

TEAM

MARIA ANTONIADOU, 5177

OLGA LIAKOU, 5267

AGGELOS MPIRINTZHS, 4741

FINAL REPORT

1 CONTENTS

2	DATABASE	5
2.1	Relational Schema at the Logical Level.....	5
2.2	Relational Schema at the PHYSICAL7 Level	7
2.2.1	Setting DBMS PARAMETERS	8
2.2.2	Configuration of the Physical Schema of the Database.....	8
2.2.3	Security Settings	16
3	Software Architecture	17
3.1.1	ETL Architecture and Structure	17
3.2	Package / Subsystem Diagrams of the Main Application.....	20
3.3	CENTRAL APP CLASS DIAGRAMS	21
4	SAMPLE QUESTIONS AND ANSWERS	31

HISTORY OF PREVIOUS EDITIONS

TEAM: (5177, 5267, 4741)

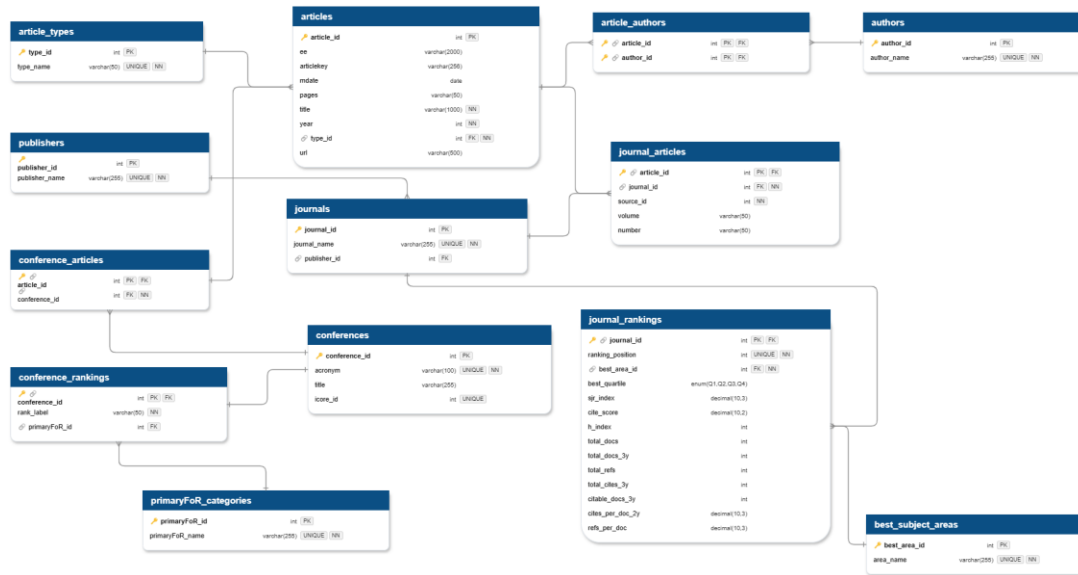
Date	Έκδοση	Description	Author
2026/03/02		Initial project setup and definition of the main application requirements.	TEAM(5177, 5267, 4741)
2026/03/05		Collection and examination of the initial datasets for articles, journals, conferences and rankings.	TEAM(5177, 5267, 4741)
2026/03/12		First design of the relational database schema and identification of the main entities.	TEAM(5177, 5267, 4741)
2026/03/13		Definition of primary keys, foreign keys and relationship tables between articles, authors, journals and conferences.	TEAM(5177, 5267, 4741)
2026/03/19		Creation of the first MySQL database schema and initial table implementation.	TEAM(5177, 5267, 4741)
2026/03/20		Development of the first ETL scripts for reading and cleaning the source data.	TEAM(5177, 5267, 4741)
2026/03/23		Implementation of data transformation steps for authors, articles, journals and conferences.	TEAM(5177, 5267, 4741)
2026/03/26		Generation of TSV files for loading the cleaned data into the database.	TEAM(5177, 5267, 4741)
2026/03/30		Implementation of the database loading process and correction of loading order due to foreign keys.	TEAM(5177, 5267, 4741)
2026/04/01		First successful loading of the main dataset into MySQL. Creation of database views for simplifying statistics and report queries.	TEAM(5177, 5267, 4741)
2026/04/02		Implementation of SQL queries for author search, author profile and author publication reports.	TEAM(5177, 5267, 4741)
2026/04/05		Implementation of SQL queries for journal search, journal profile, ranking and article reports.	TEAM(5177, 5267, 4741)
2026/04/06		Implementation of SQL queries for conference search, conference profile, ranking and article reports.	TEAM(5177, 5267, 4741)
2026/04/15		Implementation of SQL queries for year profile, available years and yearly publication reports.	TEAM(5177, 5267, 4741)
2026/04/16		Addition of batch queries for loading large article reports progressively.	TEAM(5177, 5267, 4741)

TEAM: (5177, 5267, 4741)

2026/04/17		Creation of indexes to improve search, join and filtering performance.	TEAM(5177, 5267, 4741)
2026/04/18		Setup of the Java project structure and database connection through Java.	TEAM(5177, 5267, 4741)
2026/05/19		Implementation of repository classes for executing SQL queries and mapping results.	TEAM(5177, 5267, 4741)
2026/05/20		Implementation of service classes for authors, journals, conferences and years.	TEAM(5177, 5267, 4741)
2026/05/21		Implementation of JavaFX controllers and connection with the service layer. Development of the main JavaFX screens for authors, venues, years and charts.	TEAM(5177, 5267, 4741)
2026/05/22		Addition of tables, filters and line charts in the graphical user interface. Implementation of journal and conference analysis charts, including categories and rankings.	TEAM(5177, 5267, 4741)
2026/05/23		Testing of the main application functions and correction of query and UI issues.	TEAM(5177, 5267, 4741)
2026/05/25		Final check of application functionality and report completion	TEAM(5177, 5267, 4741)

2 DATABASE

2.1 RELATIONAL SCHEMA AT THE LOGICAL LEVEL



Σχήμα 2.1 Relational schema of the system's database

```

3 ● DROP DATABASE IF EXISTS research_publications_db;
4
5 ● CREATE DATABASE research_publications_db
6 CHARACTER SET utf8mb4
7 COLLATE utf8mb4_unicode_ci;
8
9 ● USE research_publications_db;
10
11 -- =====
12 -- LOOKUP TABLE: article_types
13 -- 1 = journal
14 -- 2 = conference
15 -- =====
16
17 ● CREATE TABLE article_types (
18     type_id INT PRIMARY KEY,
19     type_name VARCHAR(50) NOT NULL UNIQUE
20 ) ENGINE=InnoDB;
  
```

TEAM: (5177,5267,4741)

```

23 -- =====
24 -- MAIN TABLE: articles
25 -- =====
26
27 CREATE TABLE articles (
28     article_id INT PRIMARY KEY,
29     ee TEXT,
30     articlekey VARCHAR(1000),
31     mdate DATE,
32     pages VARCHAR(50),
33     title VARCHAR(1000) NOT NULL,
34     year INT NOT NULL,
35     type_id INT NOT NULL,
36     url VARCHAR(500),
37
38     CONSTRAINT fk_articles_type
39     FOREIGN KEY (type_id)
40     REFERENCES article_types(type_id)
41 ) ENGINE=InnoDB;
42
43 -- =====
44 -- LOOKUP TABLE: authors
45 -- =====
46
47
48 CREATE TABLE authors (
49     author_id INT PRIMARY KEY,
50     author_name VARCHAR(255) CHARACTER SET utf8mb4
51     COLLATE utf8mb4_0900_as_cs NOT NULL,
52     UNIQUE KEY uq_authors_name (author_name)
53 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
54
55
56
57
58
59
60 CREATE TABLE publishers (
61     publisher_id INT PRIMARY KEY,
62     publisher_name VARCHAR(255) NOT NULL UNIQUE
63 ) ENGINE=InnoDB;
64
65 CREATE TABLE primaryFOR_categories (
66     primaryFOR_id INT PRIMARY KEY,
67     primaryFOR_name VARCHAR(255) NOT NULL UNIQUE
68 ) ENGINE=InnoDB;
69
70 CREATE TABLE journals (
71     journal_id INT PRIMARY KEY,
72     journal_name VARCHAR(255) NOT NULL UNIQUE,
73     publisher_id INT,
74
75     CONSTRAINT fk_journals_publisher
76     FOREIGN KEY (publisher_id)
77     REFERENCES publishers(publisher_id)
78 ) ENGINE=InnoDB;
79
80 CREATE TABLE conferences (
81     conference_id INT PRIMARY KEY,
82     acronym VARCHAR(100) NOT NULL UNIQUE,
83     title VARCHAR(255),
84     icore_id INT UNIQUE
85 ) ENGINE=InnoDB;
86
87
88
89
90
91
92
93 CREATE TABLE article_authors (
94     article_id INT NOT NULL,
95     author_id INT NOT NULL,
96
97     PRIMARY KEY (article_id, author_id),
98
99     CONSTRAINT fk_article_authors_article
100     FOREIGN KEY (article_id)
101     REFERENCES articles(article_id)
102     ON DELETE CASCADE,
103
104     CONSTRAINT fk_article_authors_author
105     FOREIGN KEY (author_id)
106     REFERENCES authors(author_id)
107     ON DELETE CASCADE
108 ) ENGINE=InnoDB;
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129 CREATE TABLE journal_articles (
130     article_id INT PRIMARY KEY,
131     journal_id INT NOT NULL,
132     source_id INT NOT NULL,
133     volume VARCHAR(50),
134     number VARCHAR(50),
135
136     CONSTRAINT fk_journal_articles_article
137     FOREIGN KEY (article_id)
138     REFERENCES articles(article_id)
139     ON DELETE CASCADE,
140
141     CONSTRAINT fk_journal_articles_journal
142     FOREIGN KEY (journal_id)
143     REFERENCES journals(journal_id)
144 ) ENGINE=InnoDB;
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999

```

TEAM: (5177,5267,4741)

```

143 ● CREATE TABLE journal_rankings (
144     journal_id INT PRIMARY KEY,
145     ranking_position INT UNIQUE NOT NULL,
146     best_area_id INT NOT NULL,
147     best_quartile ENUM('Q1','Q2','Q3','Q4'),
148     sjr_index DECIMAL(10,3),
149     cite_score DECIMAL(10,2),
150     h_index INT,
151     total_docs INT,
152     total_docs_3y INT,
153     total_refs INT,
154     total_cites_3y INT,
155     citable_docs_3y INT,
156     cites_per_doc_2y DECIMAL(10,3),
157     refs_per_doc DECIMAL(10,3),
158
159     CONSTRAINT fk_journal_rankings_journal
160     FOREIGN KEY (journal_id)
161     REFERENCES journals(journal_id),
162
163     CONSTRAINT fk_journal_rankings_area
164     FOREIGN KEY (best_area_id)
165     REFERENCES best_subject_areas(best_area_id)
166 ) ENGINE=InnoDB;
-- mysql Ver 8.0.44 for Win64 on x86_64 (MySQL Community Server - GPL)

169 ● CREATE TABLE conference_rankings (
170     conference_id INT PRIMARY KEY,
171     rank_label VARCHAR(50) NOT NULL,
172     primaryFoR_id INT,
173
174     CONSTRAINT fk_conference_rankings_conference
175     FOREIGN KEY (conference_id)
176     REFERENCES conferences(conference_id),
177
178     CONSTRAINT fk_conference_rankings_primaryFoR
179     FOREIGN KEY (primaryFoR_id)
180     REFERENCES primaryFoR_categories(primaryFoR_id)
181 ) ENGINE=InnoDB;

```

2.2 RELATIONAL SCHEMA AT THE PHYSICAL7 LEVEL

This section presents the relational schema of the database at the physical level, as implemented in MySQL. It describes the database tables, their main attributes, the primary and foreign keys, as well as the relationships between the tables.

The database includes the following tables: article_types, articles, authors, article_authors, journals, journal_articles, conferences, conference_articles, publishers, best_subject_areas, journal_rankings, primaryFoR_categories and conference_rankings.

Schema	Description
articles	Contains the basic information of the articles
authors	Contains the information of the authors
article_authors	Links articles with authors
journals	Contains the journals
journals_articles	Links articles with journals
conferences	Contains the conferences
conference_articles	Links articles with conferences
publishers	Contains the publishers
article_types	Contains the article types
journal_rankings	Contains journal rankings
conference_rankings	Contains conference rankings
best_subject_areas	Contains subject areas
primaryFoR_categories	Contains Field of Research categories

2.2.1 SETTING DBMS PARAMETERS

The database management system used for the implementation of the project was MySQL 8.0.40. The database was created using the utf8mb4 character set and the utf8mb4_unicode_ci collation. This configuration allows the database to store Unicode text correctly and supports the textual data used in articles, authors, journals and conferences.

Foreign key checks are enabled during normal execution of the database. This ensures referential integrity between the related tables, such as articles, authors, journals, conferences and their relationship tables.

Regarding memory-related DBMS parameters, the MySQL configuration uses an InnoDB buffer pool size of 2 GB, a join buffer size of 256 KB, a sort buffer size of 256 KB, a maximum heap table size of 16 MB and a temporary table size of 42 MB. These settings were not manually tuned for the project, since the application is mainly read-oriented and query performance was improved mainly through the use of indexes, views and organized SQL queries.

2.2.2 CONFIGURATION OF THE PHYSICAL SCHEMA OF THE DATABASE

Data access optimization was achieved through the use of indexes, views, and organized SQL queries..

INDEXES

Indexes were created in the main tables of the database in order to improve the performance of the most frequent queries of the application. The application performs many searches, joins, groupings and filters based on years, article types, authors, journals and conferences. Therefore, indexes were added mainly on foreign keys, search fields and frequently filtered columns.

In the articles table, indexes were created on the year and type_id columns. These indexes support queries related to year profiles, publication reports, line charts and filtering articles by publication type and year. Composite indexes such as (type_id, year), (year, type_id), (year, article_id) and (type_id, year, article_id) were also created to support queries that combine multiple filtering conditions.

In the relationship tables article_authors, journal_articles and conference_articles, indexes were created on both sides of the relationships. These tables are frequently used in joins between articles and authors, journals or conferences. For example, indexes on author_id and article_id help the system retrieve all publications of a specific author, count distinct authors per article, and calculate author statistics. Similarly, indexes on journal_id and conference_id help retrieve articles and statistics for a selected journal or conference.

TEAM: **(5177, 5267, 4741)**

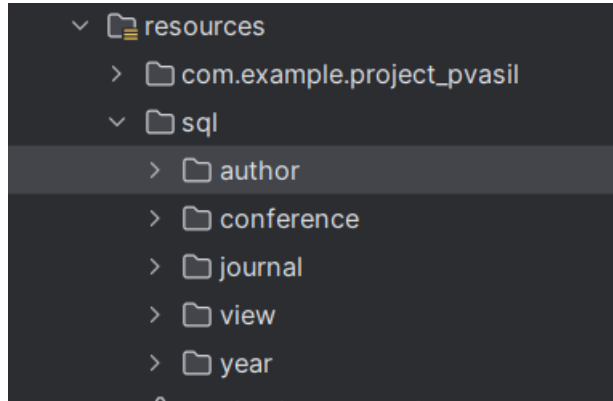
Indexes were also created on search fields, such as author_name, journal_name, conference_title and acronym. These indexes support the search functionality of the graphical user interface, where the user can search for authors, journals and conferences. These indexes are especially useful when the search pattern starts with the given text.

Finally, indexes were added to the ranking tables, such as journal_rankings and conference_rankings, on the columns that are used for joins and lookups. These indexes improve the retrieval of ranking information, subject areas and primary FoR categories.

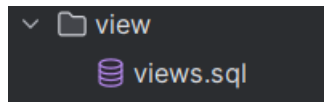
After creating the indexes, ANALYZE TABLE commands were executed. This allows MySQL to update table statistics and helps the query optimizer choose more efficient execution plans.

Table	Indexes	Purpose
articles	year, type_id, year, year, type_id, year, article_id, type_id, year, article_id	Used for year filters, article type filters, reports and line charts
article_authors	article_id, author_id, author_id, article_id, article_id, author_id	Used for author profiles, author publications and counting authors per article
journal_articles	journal_id, article_id, journal_id, article_id	Used for journal profiles, journal yearly statistics and journal article reports
conference_articles	conference_id, article_id, conference_id, article_id	Used for conference profiles, conference yearly statistics and conference article reports
authors	author_name	Used for author search
journals	journal_name, publisher_id	Used for journal search and joins with publishers
conferences	acronym, conference_title	Used for conference search
journal_rankings	journal_id, best_subject_area_id	Used for journal ranking lookup and subject area analysis
conference_rankings	conference_id, primaryFoR_id	Used for conference ranking lookup and Primary FoR analysis

The SQL queries were organized based on the main requirements of the application, namely author profile, conference/journal profile, year profile, article reports, and data for charts. The logic for calculating statistics was implemented mainly within the DBMS, using JOIN, GROUP BY, COUNT, COUNT DISTINCT, MIN, MAX, AVG, and time-range filters. In this way, transferring all data to the application's main memory and processing it in Java code is avoided.



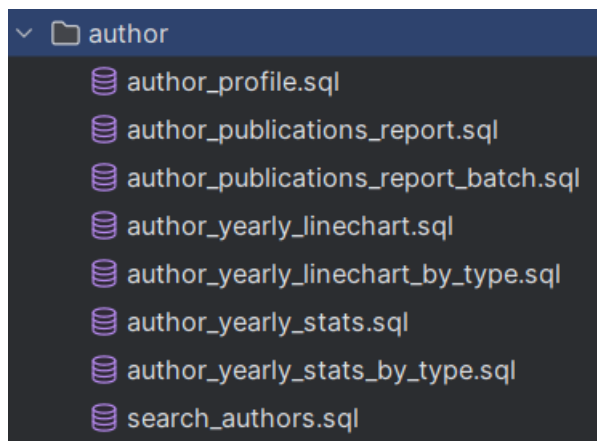
The **views.sql** file contains the main views of the database. These views were created to simplify the application's queries and to group complex calculations, such as the number of articles, the number of authors, averages, rankings, and data for charts.



View	What it implements
vw_article_authors_agg	Aggregates the authors of each article into a single row
vw_publication_report	Unified report for all articles, whether they are journal or conference articles
vw_journal_year_stats	Journal statistics per year
vw_conference_year_stats	Conference statistics per year
vw_journal_profile_stats	Aggregated journal profile
vw_conference_profile_stats	Aggregated conference profile
vw_year_profile_stats	Aggregated year profile

View	What it implements
vw_author_profile_stats	Aggregated author profile
vw_author_year_stats	Author statistics per year
vw_journal_ranking_metrics	Ranking and bibliometric information of journals
vw_conference_ranking_metrics	Ranking and Primary FoR information of conferences
vw_journal_area_year_stats	Statistics by Best Subject Area and year
vw_conference_for_year_stats	Statistics by Primary FoR and year
vw_publisher_journal_counts	Number of journals per publisher
vw_publisher_quartile_counts	Number of journals per publisher and quartile
vw_journal_year_scatter_stats	Data for journal scatter plots
vw_conference_year_scatter_stats	Data for conference scatter plots

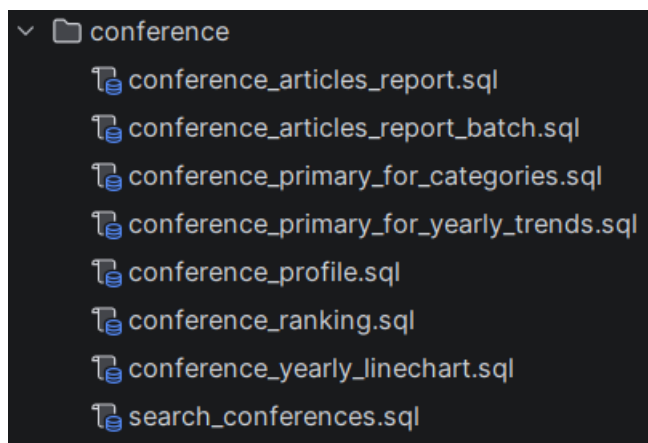
The author package includes the SQL queries that implement the application's functions related to authors. Specifically, it covers author search, displaying the author's profile, generating a detailed report of their publications, and extracting data for line charts per year.



SQL script	What it implements
search_authors.sql	Searches for authors based on part of their name
author_profile.sql	Calculates the first/last year, total number of articles, active years, and average number of

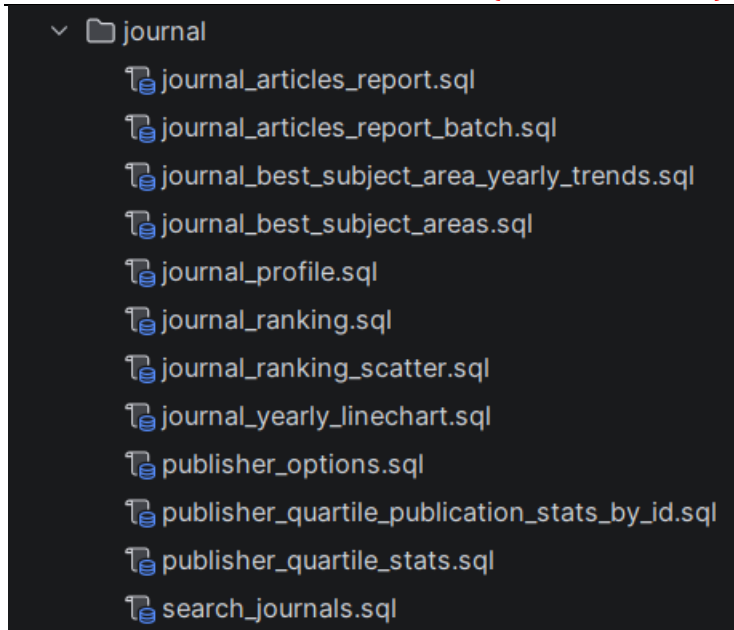
	articles per year
author_publications_report.sql	Returns the detailed articles of an author for a specific range of years
author_publications_report_batch.sql	Returns an author's publications in batches, using last_article_id and batch_size, so that all results are not loaded at once
author_yearly_linechart.sql	Returns the number of articles of the author per year
author_yearly_stats.sql	Calculates the author's annual statistics directly from the article_authors and articles tables, namely the number of distinct articles per year
author_yearly_linechart_by_type.sql	Returns articles per year divided into journals and conferences
author_yearly_stats_by_type.sql	Calculates the author's annual statistics by publication type, namely total articles, journal articles, and conference articles per year.

To support the functions related to conferences, the conference SQL files package was created. The queries in this package cover conference search, conference profile statistics, ranking information, yearly publication statistics, detailed article reports, batch loading of articles, and analysis based on the Primary For categories.



SQL script	What it implements
search_conferences.sql	Searches for conferences based on acronym or title, so that the user can select a conference from the UI. The query also supports a maximum number of results for autocomplete/search functionality.
conference_profile.sql	Calculates the main statistics of a selected conference for a given year range, such as first and last publication year, active years, total articles, total author occurrences, distinct authors, average articles per year and average authors per article.
conference_ranking.sql	Returns the ranking information of a selected conference, including its acronym, title, iCORE id, rank label and Primary FoR category.
conference_yearly_linechart.sql	Returns annual statistics for the conference, such as the number of articles, total author appearances, and distinct authors, for use in a line chart
conference_articles_report.sql	Returns the detailed publications of the conference
conference_articles_report_batch.sql	Returns the publications of a conference in batches, using last_article_id and batch_size, so that all articles are not loaded at once.
conference_primary_for_categories.sql	Returns the available Primary FoR categories, including their id and name, so that they can be used as filters in the UI.
conference_primary_for_yearly_trends.sql	Returns yearly trends for conferences grouped by Primary FoR category. It counts the number of distinct conferences per category and year, with optional filtering by Primary FoR category and year range.

To support the functions related to journals, the journal SQL files package was created. The queries in this package cover journal search, journal profile statistics, ranking information, yearly publication statistics, detailed article reports, batch loading of articles, ranking scatter plots, Best Subject Area analysis and publisher-based quartile statistics.

**SQL script****What it implements**

search_journals.sql

Searches for journals based on their name, so that the user can select a journal from the UI. The query also returns publisher information and supports a maximum number of results for autocomplete/search functionality.

journal_profile.sql

Calculates the main statistics of a selected journal for a given year range, such as first and last publication year, active years, total articles, total author occurrences, distinct authors, average articles per year and average authors per article.

journal_ranking.sql

Returns the ranking information of a selected journal, including publisher, ranking position, best quartile, SJR index, CiteScore, h-index, document/citation indicators and best subject area.

journal_yearly_linechart.sql

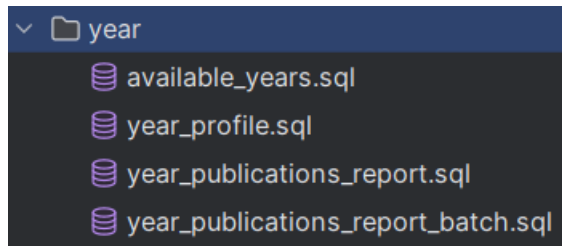
Returns yearly statistics for a selected journal, such as total articles, total author occurrences, distinct authors and average authors per article, for use in a line chart.

journal_articles_report.sql

Returns the detailed publication report of a selected journal for a specific year range, including article information,

SQL script	What it implements
	journal information and aggregated author information.
journal_articles_report_batch.sql	Returns the publications of a selected journal in batches, using last_article_id and batch_size, so that all articles are not loaded at once.
journal_best_subject_areas.sql	Returns the available Best Subject Areas, including their id and name, so that they can be used as filters in the UI.
journal_best_subject_area_yearly_trends.sql	Returns yearly trends for journals grouped by Best Subject Area. It counts the number of distinct journals per subject area and year, with optional filtering by subject area and year range.
journal_ranking_scatter.sql	Returns data for scatter plots based on journal ranking metrics. Each row represents one journal, and the selected ranking indicators are used as the x-axis and y-axis values.
publisher_options.sql	Returns available publishers with the number of related publications, supporting search by publisher name and a maximum number of results for UI selection.
publisher_quartile_stats.sql	Returns publication counts for a selected publisher grouped by journal quartile, including Q1, Q2, Q3 and Q4.
publisher_quartile_publication_stats_by_id.sql	Returns journal counts for a selected publisher grouped by quartile. The query keeps the same aliases as the publication statistics query in order to remain compatible with the existing Java code.

To support the functions related to years, the year SQL files package was created. This package covers the selection of available years, displaying a year profile, and generating a detailed publication report for a specific year.



SQL script	What it implements
available_years.sql	Returns all years that exist in the database along with basic article counts
year_profile.sql	Returns aggregated statistics for a specific year
year_publications_report.sql	Returns the publications of a year with filters
year_publications_report_batch.sql	Returns the publications of a year in batches, with filters for publication type, journal, conference, and author, so that all articles are not loaded at once.

2.2.3 SECURITY SETTINGS

The application is mainly read-oriented, since users retrieve publication statistics, profiles, reports and chart data without modifying the database contents. For this reason, the recommended security configuration is based on role separation. The JavaFX application should connect to the database through an application user with SELECT privileges only. This user can read the required tables and views but cannot insert, update or delete data.

Database creation, schema modification, index creation and data loading are administrative operations and should be executed only by an administrator or loader user. In this way, normal application execution is separated from database maintenance tasks.

3 SOFTWARE ARCHITECTURE

3.1.1 ETL ARCHITECTURE AND STRUCTURE

The application's ETL process was implemented using Python scripts and aimed to transform the original data files into clean TSV files, suitable for loading into the relational schema of the database. The flow follows the three main ETL stages: extracting data from the original sources, transforming/cleaning the data, and producing final files for loading into the MySQL tables.

The overall process is executed through the `run_full_etl.py` script, which runs the individual ETL scripts in a specific order: `etl_journal_rankings.py`, `etl_conferences.py`, `etl_journals.py`, and `etl_conference_proceedings.py`.

Then, it checks whether the final TSV files have been produced, such as `articles.tsv`, `authors.tsv`, `journals.tsv`, `journal_rankings.tsv`, `conferences.tsv`, `conference_articles.tsv`, and `conference_rankings.tsv`.

Finally, it loads the data into the database through the `project_database_dump.sql` file.

The following table briefly presents the main scripts of the ETL process. The detailed description of the steps is provided afterwards, divided into the Extract, Transform, and Load stages.

ETL script	Role
<code>CSV_to_TSV_Converter.py</code>	Converts CSV files into TSV format
<code>etl_journal_rankings.py</code>	Creates journals, publishers, subject areas, and rankings
<code>etl_conferences.py</code>	Creates conferences, conference rankings, and Primary FoR categories
<code>etl_journals.py</code>	Processes journal articles and maps them to journals
<code>etl_conference_proceedings.py</code>	Processes conference articles and maps them to conferences
<code>run_full_etl.py</code>	Executes all ETL scripts in the correct order
<code>project_database_schema.sql</code>	Creates the database schema
<code>project_database_dump.sql</code>	Loads the TSV files into MySQL

In the **Extract** stage, the initial data files containing information about articles, authors, journals, conferences, and rankings were read. These files were in CSV format, and an auxiliary script was used to convert them from CSV to TSV.

The `CSV_to_TSV_Converter.py` script detects the delimiter of the original CSV file and produces the corresponding TSV file, so that all subsequent ETL steps work with a common data format.

In the **Transform** stage, data cleaning, normalization, and mapping to the relational schema of the database were performed.

For journals, the `etl_journal_rankings.py` script processes the ranking data, creates unique publishers, journals, subject areas, and ranking records. It also converts numerical fields, such as SJR, CiteScore, h-index, total documents, citations, and references, into a format suitable for insertion into the database.

For conferences, the `etl_conferences.py` script uses the iCORE file, creates unique conferences based on the acronym, and stores the `icore_id`, the ranking, and the corresponding Primary FoR category. It also keeps statistics about duplicates, missing values, and conflicts in the data.

For journal articles, the `etl_journals.py` script reads the articles and maps them to the journals. It includes mechanisms for normalizing journal titles, using aliases, matching with acronyms, prefix matching, and other techniques so that the journal names appearing in the articles are matched as accurately as possible with the official journals. At the same time, it creates the authors, articles, and the `article_authors` and `journal_articles` relationships.

For conference articles, the `etl_conference_proceedings.py` script processes the inproceedings, adds articles of type conference, separates authors, creates new authors where necessary, and maps the booktitle to the appropriate conference. The matching is performed using title and acronym normalization, exact matching, containment matching, and token matching.

In the Load stage, the final cleaned data were stored in TSV files, which correspond to the final tables of the database. The files are generated in an order that respects the dependencies of the relational schema. First, auxiliary tables are created, such as publishers, `best_subject_areas`, and `primaryFoR_categories`. Then, the main entities are created, such as journals, conferences, articles, and authors. Finally, the relationship and ranking tables are created, such as `article_authors`, `journal_articles`, `conference_articles`, `journal_rankings`, and `conference_rankings`.

The `run_full_etl.py` script executes the scripts in a specific order and, at the end, checks whether all the expected final TSV files exist.

After the final TSV files are produced by the Python ETL scripts, the database is created through the `project_database_schema.sql` file. This file defines the `research_publications_db` database, the application tables, the primary keys, the unique keys, and the foreign key constraints.

Data loading into MySQL is performed through the `project_database_dump.sql` file. This file initially temporarily disables foreign key and unique key checks, clears the tables using `TRUNCATE TABLE`, and then loads the data from the TSV files using `LOAD DATA INFILE` commands.

For each file, the tab character is defined as the field delimiter, the first header line is ignored, and values are cleaned using `TRIM` and `NULLIF`, so that empty values are stored as `NULL`.

Loading order:

article_types

authors

publishers

best_subject_areas

primaryFoR_categories

conferences

journals

articles

journal_rankings

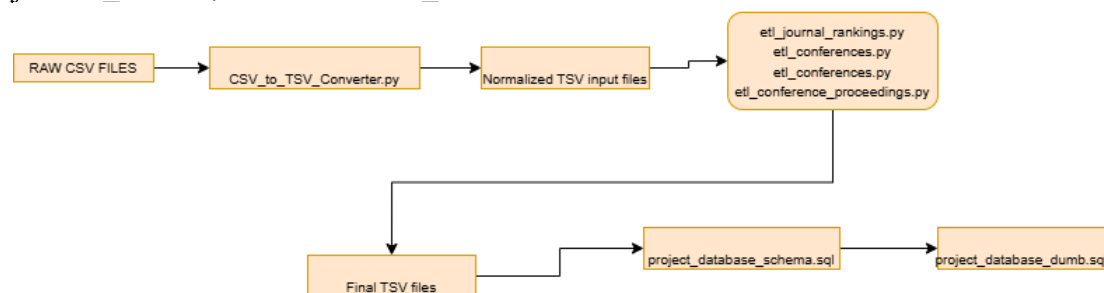
conference_rankings

article_authors

journal_articles

conference_articles

The loading order is important because the database contains foreign keys. First, the auxiliary tables are loaded, such as article_types, publishers, best_subject_areas, and primaryFoR_categories. Then, the main entities are loaded, such as authors, conferences, journals, and articles. Finally, the ranking and relationship tables are loaded, such as journal_rankings, conference_rankings, article_authors, journal_articles, and conference_articles.

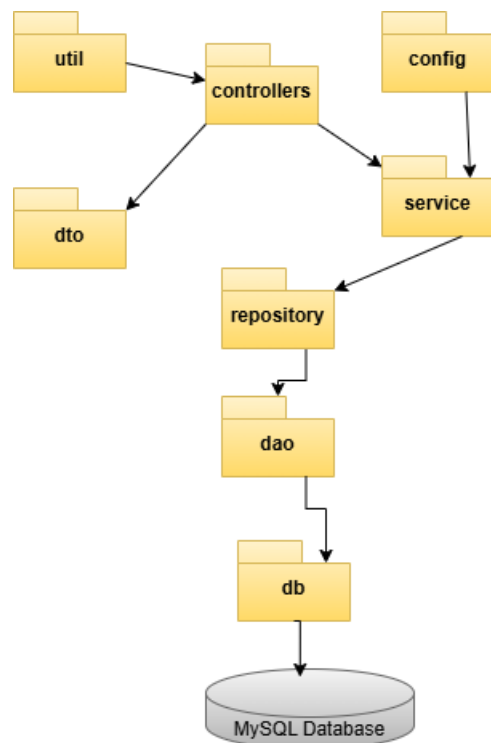


Schema 3.1 Flow of the ETL process for creating and loading the data into the MySQL database.

3.2 PACKAGE / SUBSYSTEM DIAGRAMS OF THE MAIN APPLICATION

Package diagram of the main application. The application is organized into layers in order to separate the graphical user interface from the application logic and database access. The controllers handle user actions and call the service package, where the main logic of the application is located. The service communicates with the repository, which is responsible for retrieving data and uses the SQL queries from the query package. The repository and db packages are used for executing the queries and connecting to the MySQL database.

The results are returned to the graphical user interface through dto objects, while the model package is used to represent the main entities of the application. The config and util packages provide configuration settings and helper functions that are used by the other subsystems.

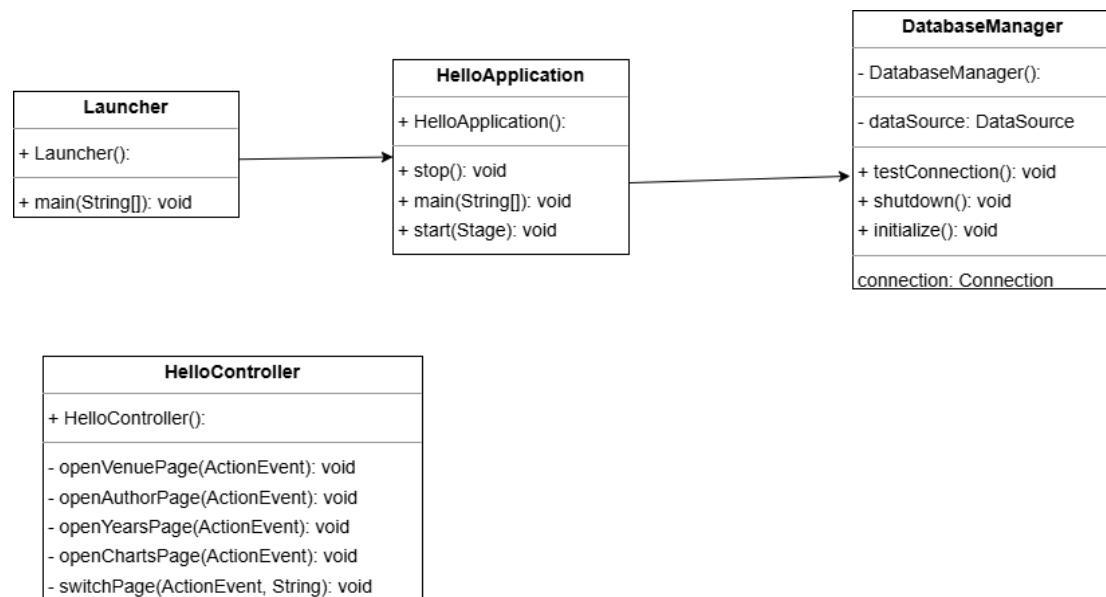


Schema 3.1 Deployment diagram of the system

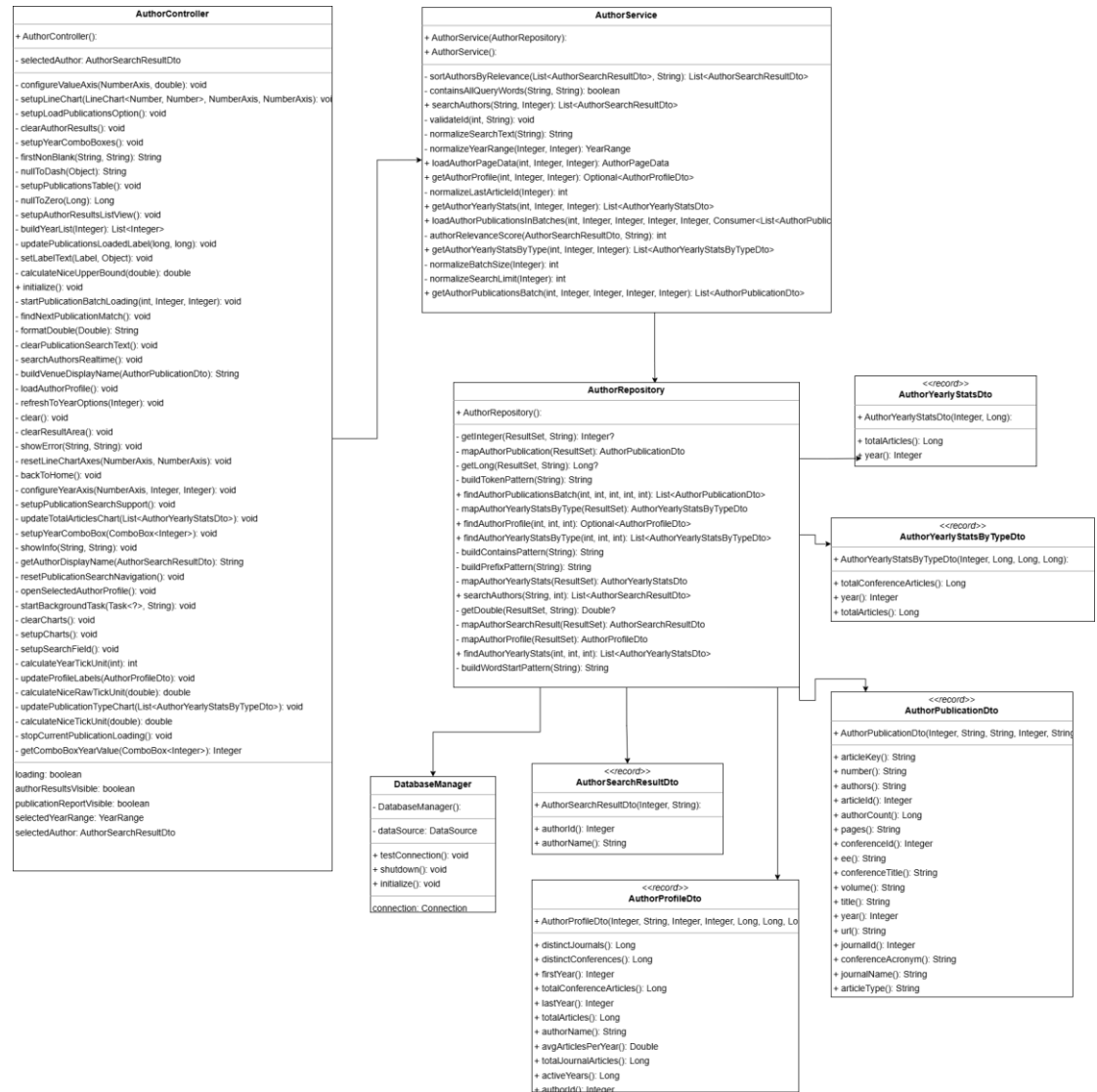
3.3 CENTRAL APP CLASS DIAGRAMS

This section presents the main class diagrams of the JavaFX application. The diagrams focus on the most important classes of the system and show how the controllers, services, repositories, DTOs and database-related classes cooperate. The application follows a layered object-oriented structure, where the user interface communicates with controller classes, the controllers call services, and the services retrieve data through repository classes.

Class diagram of the application startup and database initialization.



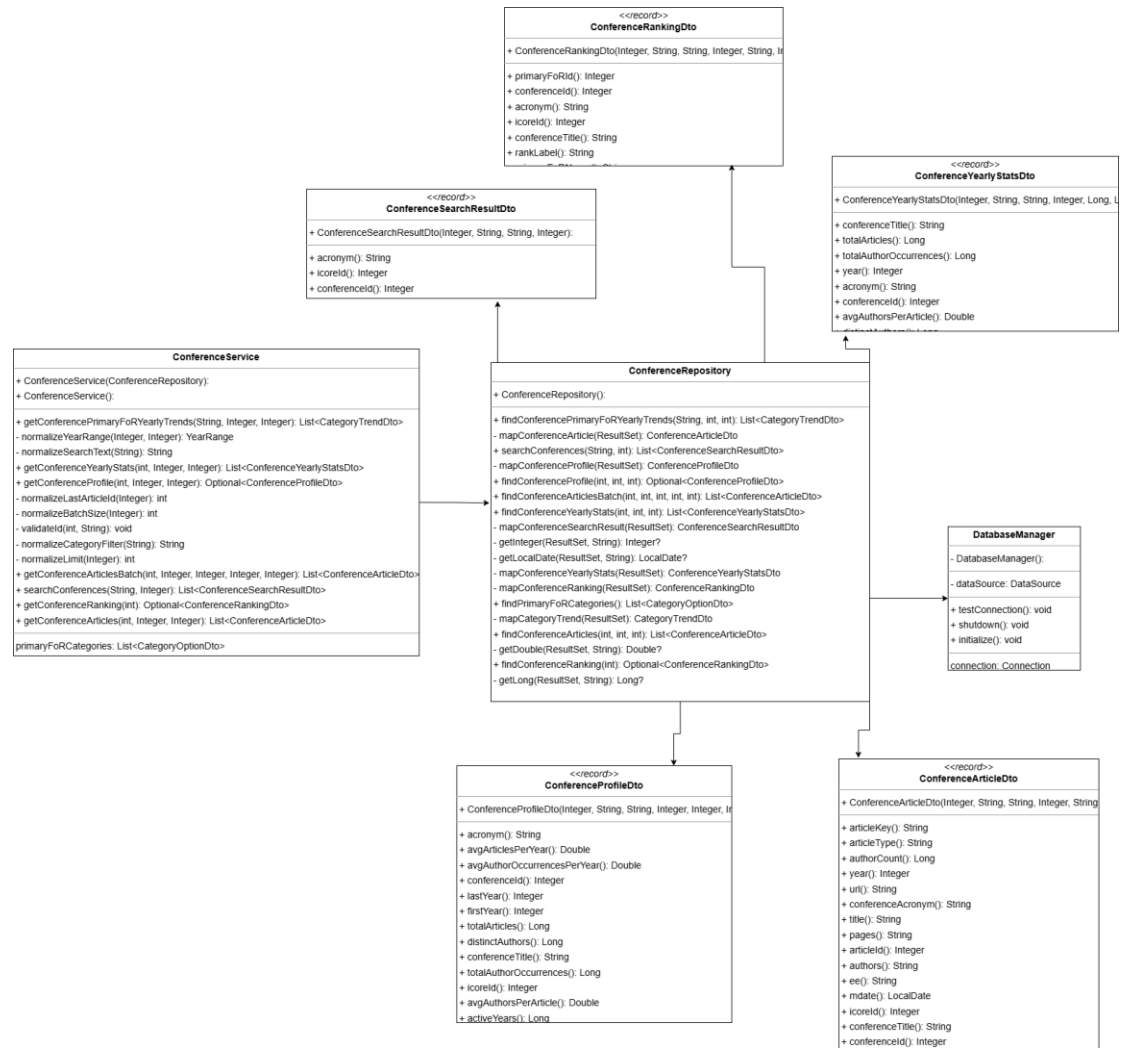
Class diagram of the author module.



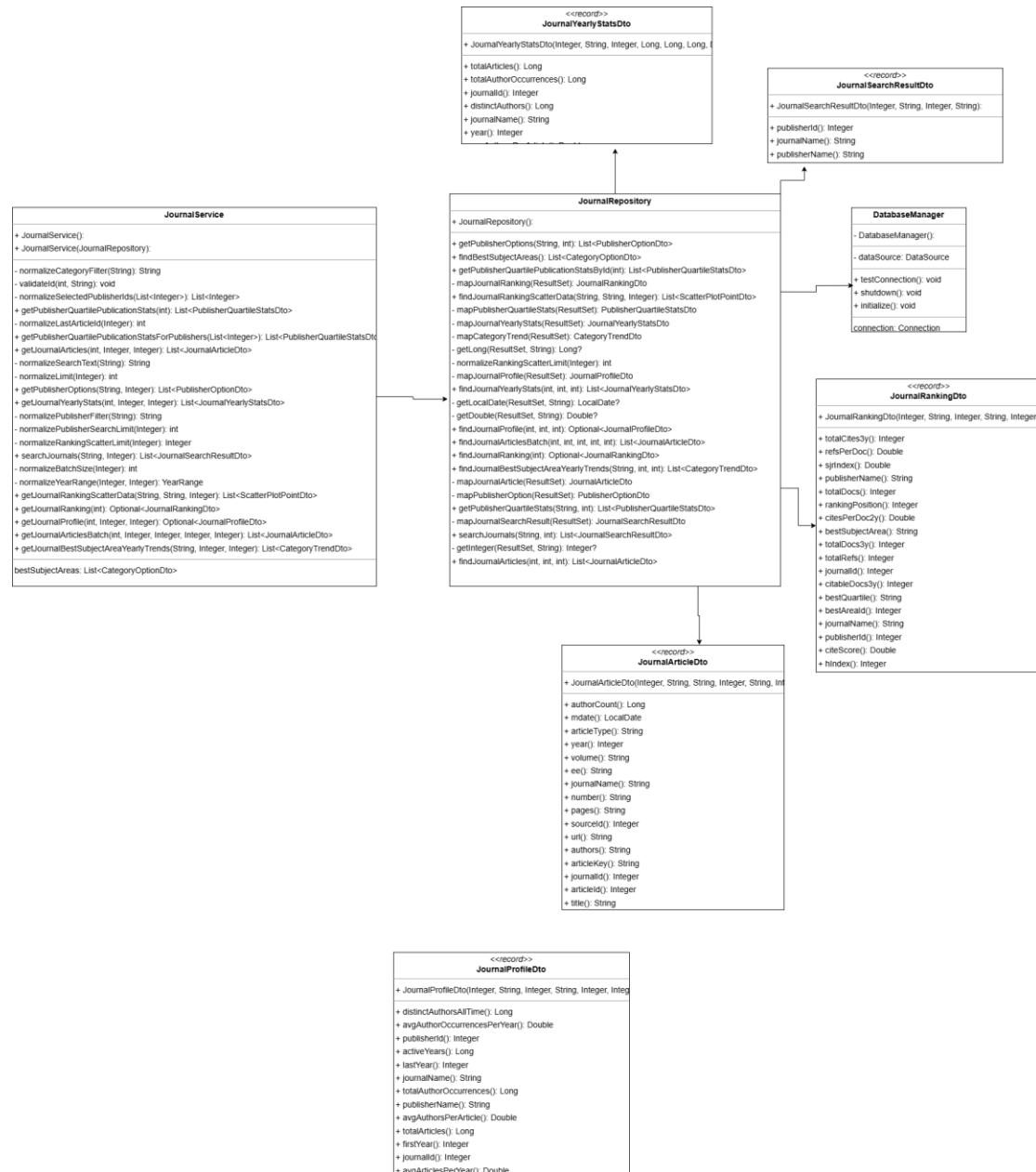
Class diagram of the venue module.



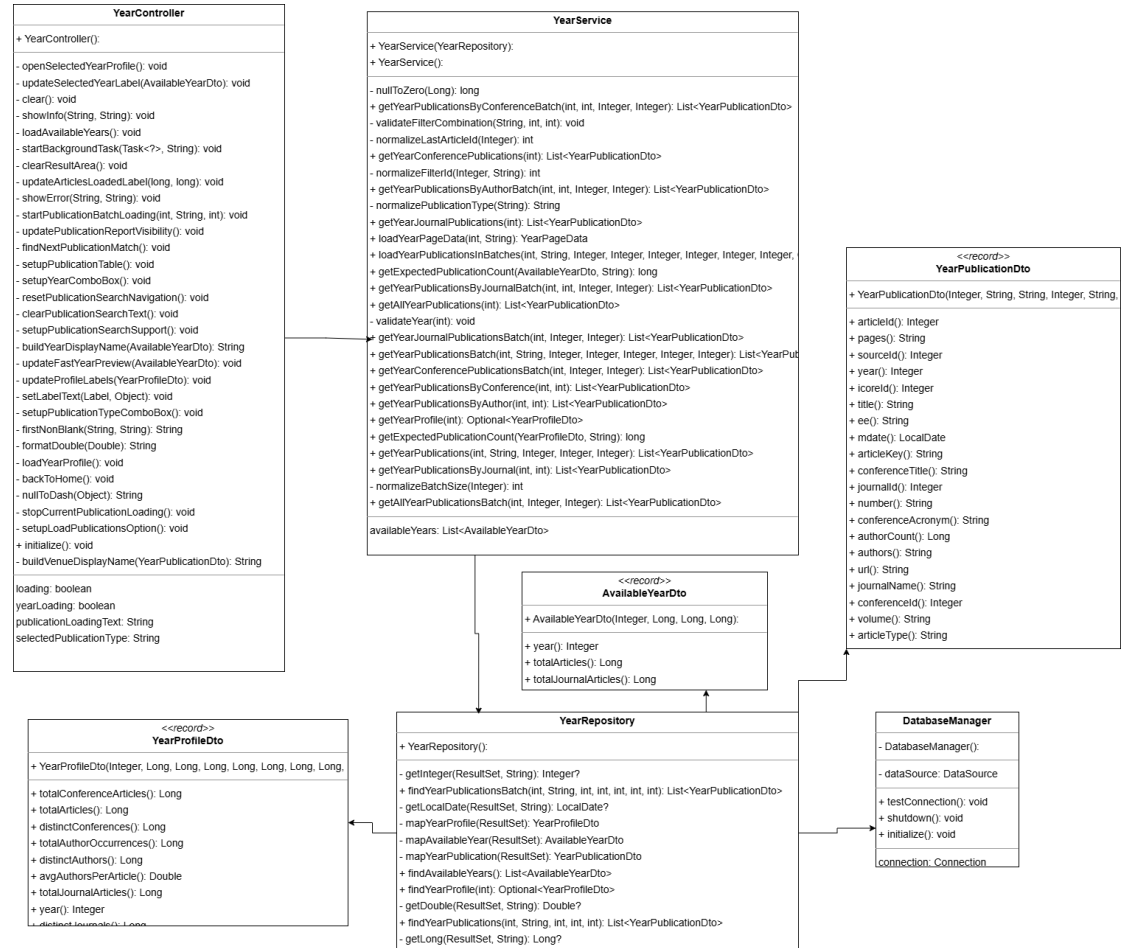
Class diagram of the conference module.



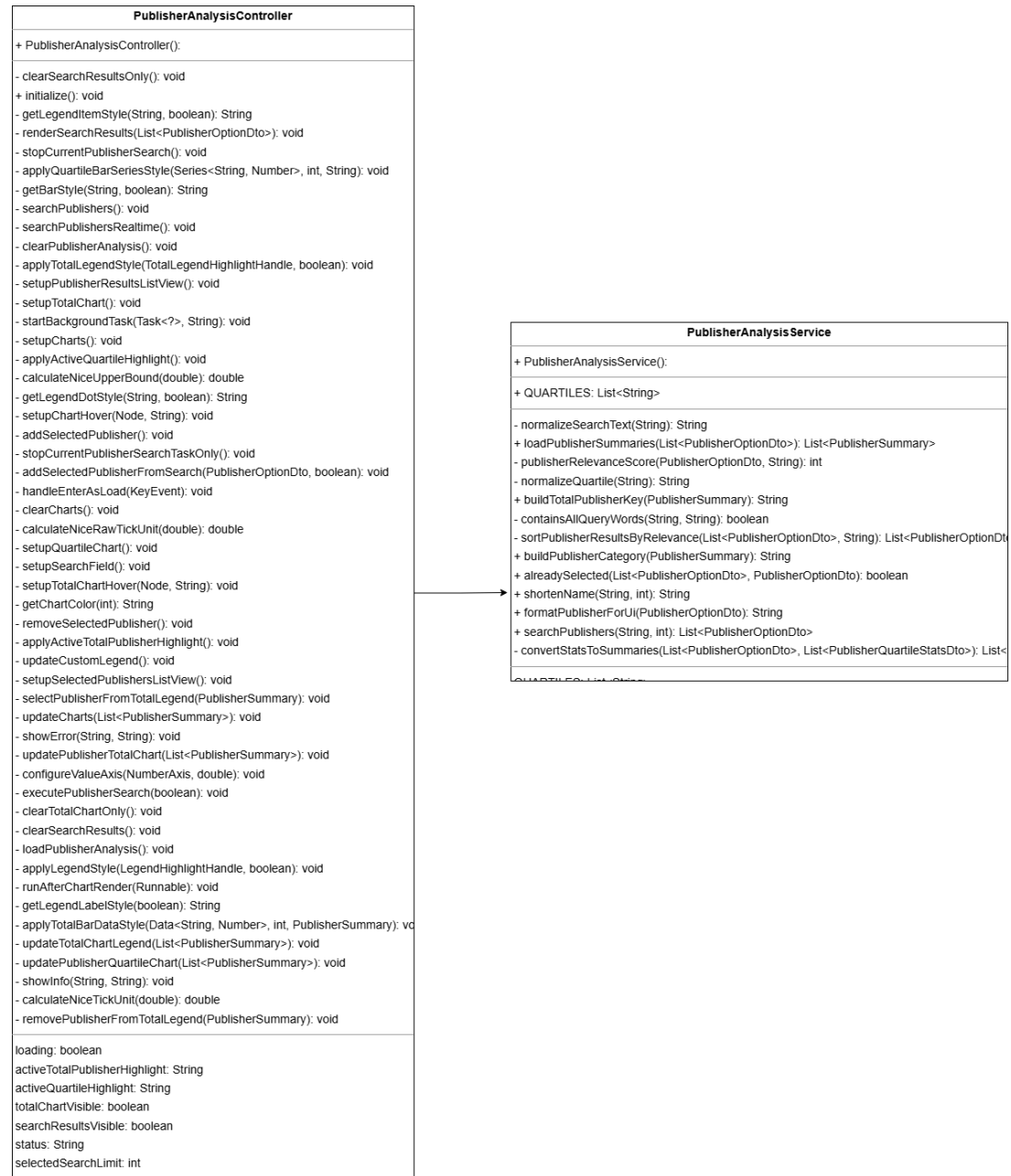
Class diagram of the journal module.



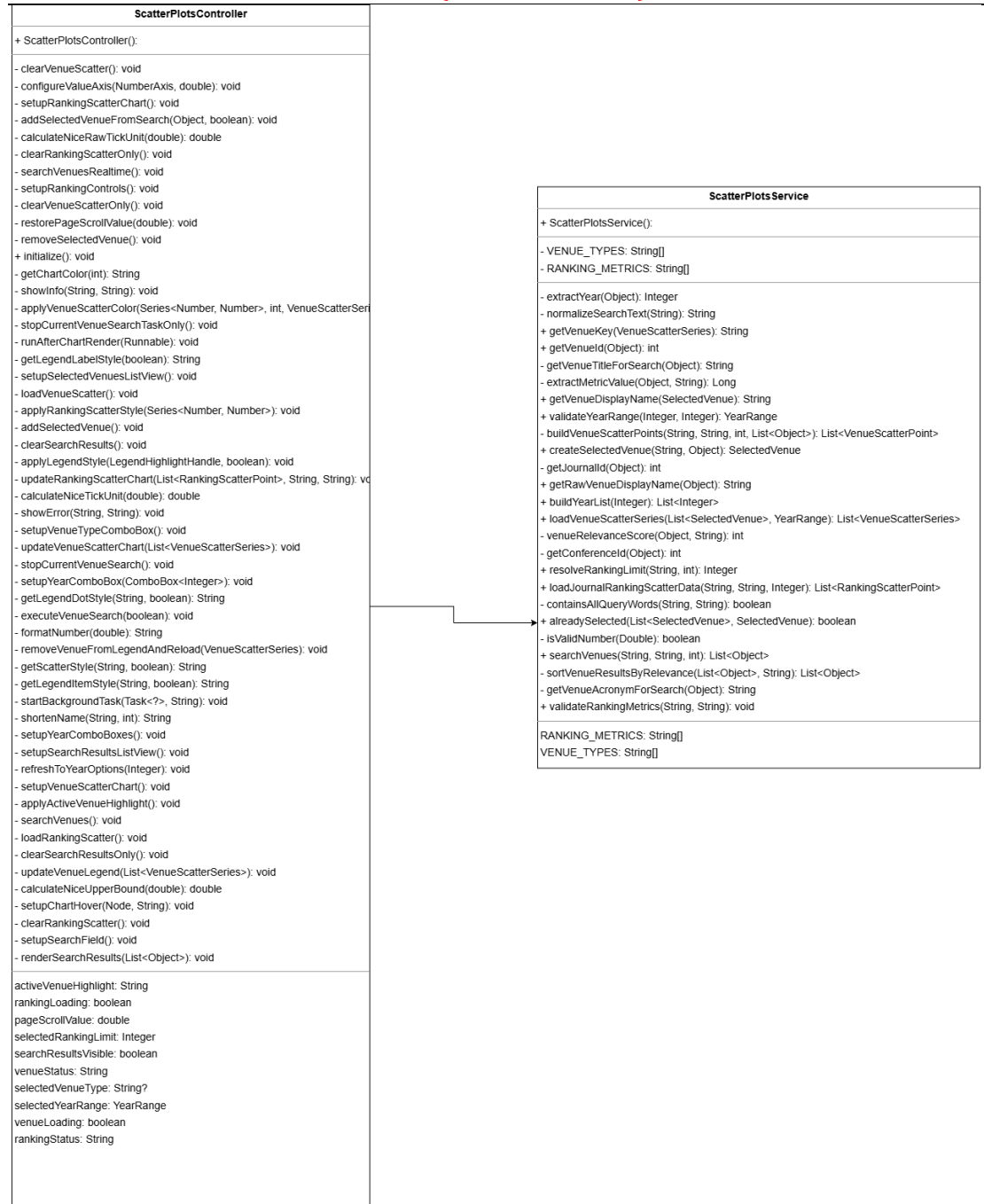
Class diagram of the year module.

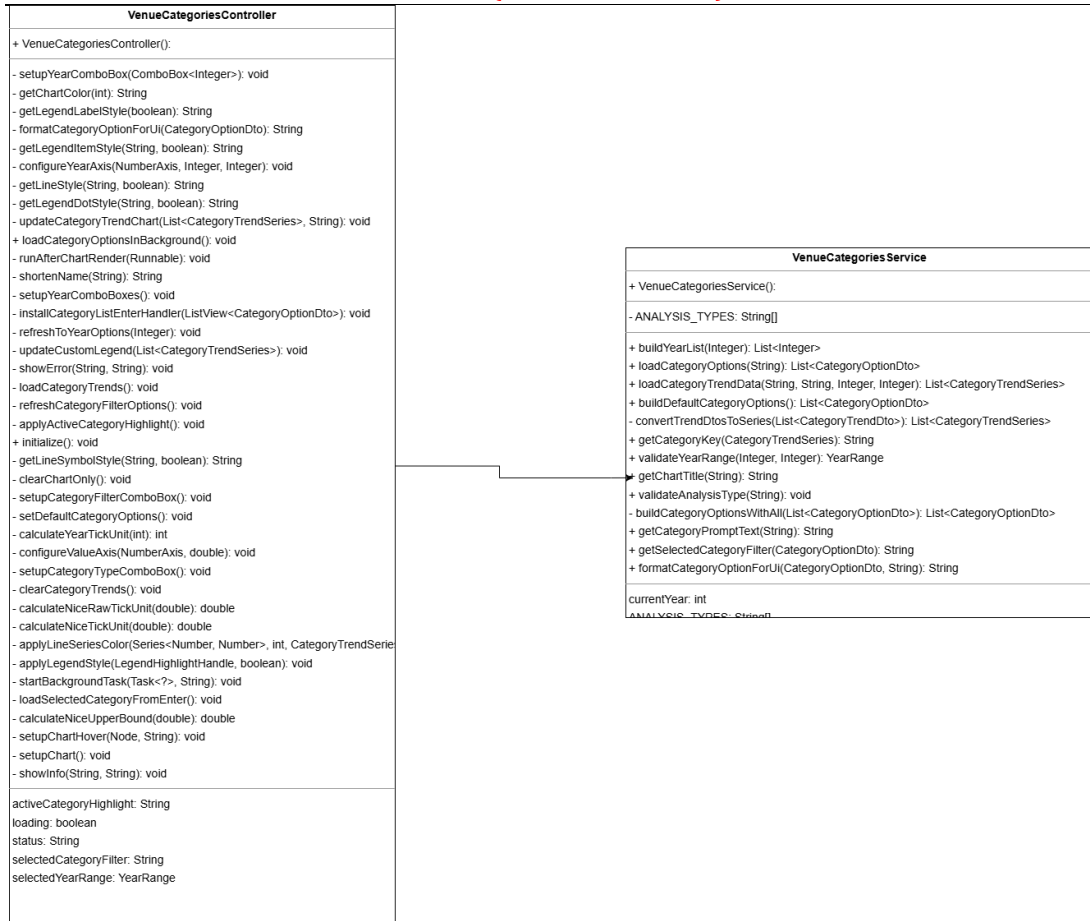


Class diagram of the charts module.



TEAM: (5177, 5267, 4741)





TEAM: (5177, 5267, 4741)



The above class diagrams show the internal organization of the main JavaFX application. The controller classes handle user interaction and forward requests to the service layer. The service classes contain the main application logic and communicate with the repository layer. The repository classes execute SQL queries through the database connection provided by DatabaseManager and map the results to DTO objects. These DTOs are then returned back to the controllers and displayed in the graphical user interface.

4 SAMPLE QUESTIONS AND ANSWERS

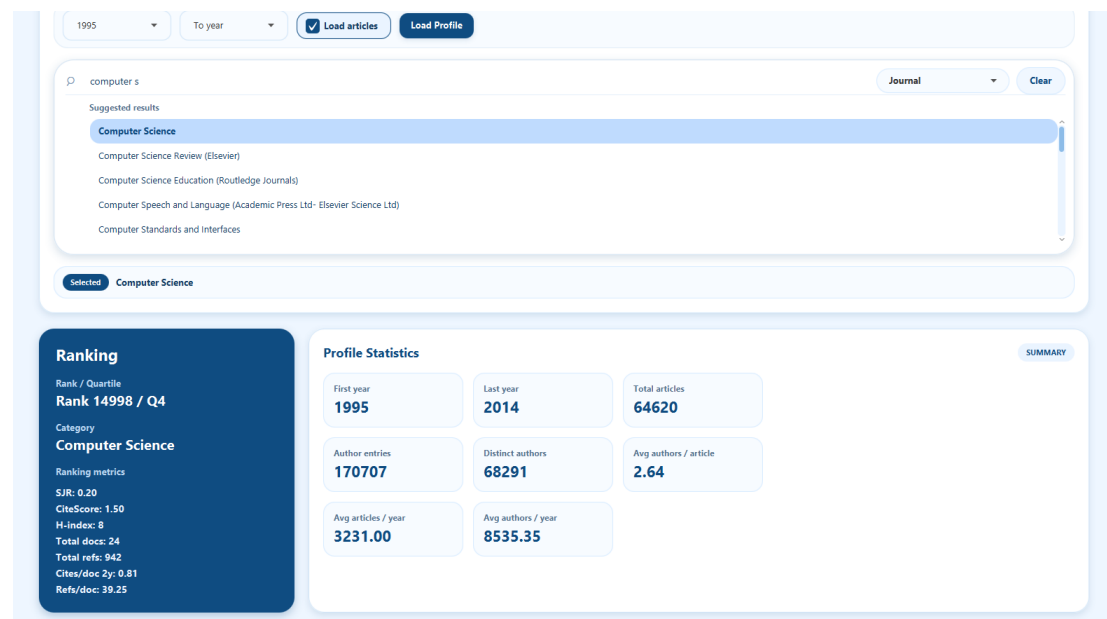
Example 1: Venue Statistics Search

Question:

What are the publication statistics and yearly activity of the journal “Computer Science”?

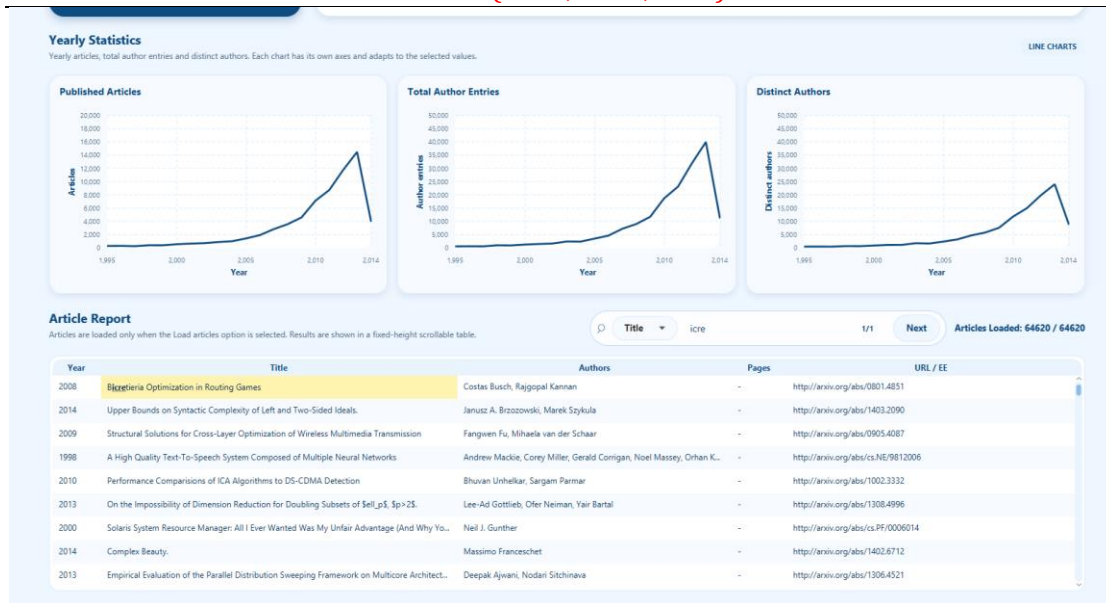
User actions:

The user starts from the home page and selects the Journals / Conferences section. Then, the user selects Journal as the venue type and searches for “computer s”. From the suggested results, the user selects “Computer Science”. The user loads the profile statistics and optionally loads the related articles.



Answer / Output:

The application displays the ranking information of the selected journal, including its rank, quartile, category and ranking metrics. It also displays profile statistics such as first year, last year, total articles, author entries, distinct authors, average authors per article, average articles per year and average authors per year. In addition, the application generates yearly line charts for published articles, total author entries and distinct authors. Finally, the article report table presents the publications of the selected journal, including year, title, authors, pages and URL/EE.



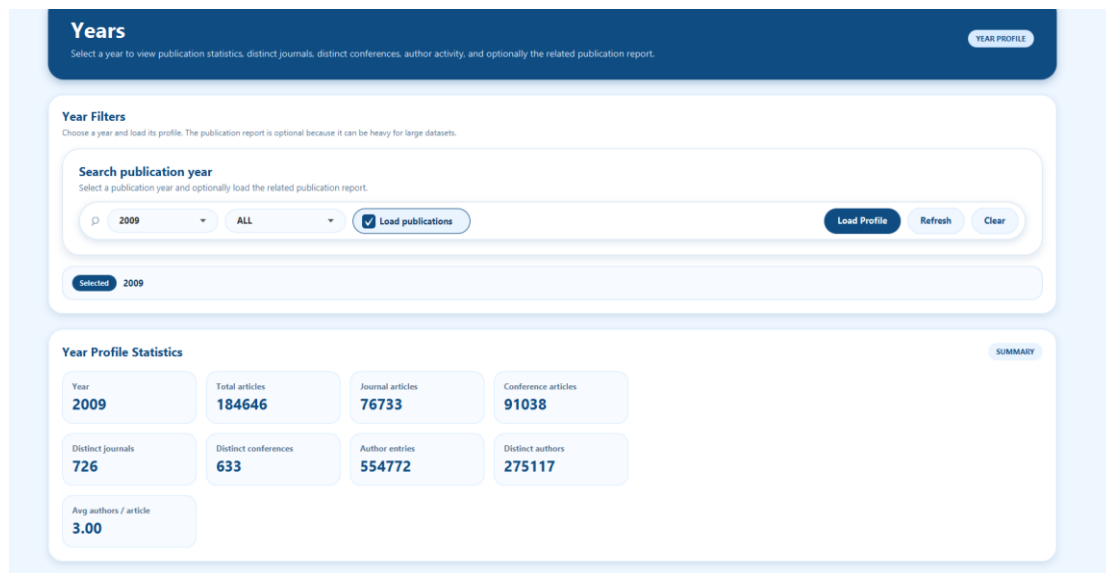
Example 2: Year Search and Publication Report

Question:

What publications and statistics are available for the year 2009?

User actions:

The user selects the Years section, chooses the year 2009 and enables the Load publications option. Then, the user presses Load Profile.



Answer / Output:

The system displays aggregated statistics for 2009, including total articles, journal articles, conference articles, distinct journals, distinct conferences, author entries, distinct authors and average authors per article. It also loads a publication report table

TEAM: (5177, 5267, 4741)

with the articles of the selected year. The user can search within the table, for example by author name, title or venue, and navigate through the results.

Publication Report
Publications are loaded only when the Load publications option is selected.

Search: Au... yang liu 1/121 Next Articles Loaded: 184646 / 184646

Year	Type	Title	Venue	Authors	Pages	URL / EE
2009	conference	A Conceptual Model for Studying the Influence of Charismatic Leadership o...	HIC - Health Informatics Conference	Qing Hu, Yang Liu , Yuqiang Feng, Zhen Shao	1-9	http://dx.doi.org/10.1109/HI...
2009	conference	Lazy User Theory: A Dynamic Model to Understand User Selection of Produ...	HIC - Health Informatics Conference	Franck Tétard, Mikael Collan	1-9	http://dx.doi.org/10.1109/HI...
2009	conference	Conference Administration.	HIC - Health Informatics Conference	NO AUTHORS	1-11	http://dx.doi.org/10.1109/HI...
2009	conference	Project Management Strategies for Prototyping Breakdowns.	HIC - Health Informatics Conference	Jan Pries-Heje, Maren Sander Granlien, Richard Baskerville	1-10	http://dx.doi.org/10.1109/HI...
2009	conference	When Is It Beneficial for a Firm to Pursue a Unified Procurement Strategy for...	HIC - Health Informatics Conference	Juliana Y. Tsai, Robert J. Kauffman	1-10	http://dx.doi.org/10.1109/HI...
2009	conference	Empirical Estimates and Observations of ODay Vulnerabilities.	HIC - Health Informatics Conference	May R. Chaffin, Miles A. McQueen, Trevor A. McQueen, ...	1-12	http://dx.doi.org/10.1109/HI...
2009	conference	The Opening "Black Box" between Conflict and Knowledge Sharing: A Psych...	HIC - Health Informatics Conference	Chuanjie Guo, Xi Zhang, Zhenjiao Chen	1-10	http://dx.doi.org/10.1109/HI...
2009	conference	Design of New Business Models for Service Integrators by Creating Informat...	HIC - Health Informatics Conference	Henrik Ickler, Reinhard Jung, Sandro Georgi, Ulrike Baumöl	1-10	http://dx.doi.org/10.1109/HI...
2009	conference	Determinants of Outsourcing Success in the Financial Industry: The Impact o...	HIC - Health Informatics Conference	Branimir Brodnik, Clemens Gunne, Daniel Beinborn, Seb...	1-10	http://dx.doi.org/10.1109/HI...

Example 3: Author Search and Author Profile

Question:

What information is available for the author H. Vincent Poor?

User actions:

The user opens the Authors section, types "h vincent p" in the search field and selects H. Vincent Poor from the suggested results. Then, the user optionally selects a year range and enables the Load articles option. After pressing Load Profile, the application retrieves the author's statistics and publication data.

1940 To year Load articles Load Profile

h vincent p Clear

Suggested results

- H. Vincent Poor**
- Harold Vincent Poor
- Joseph Vincent Rispoli

Selected H. Vincent Poor To year must be ≥ From year

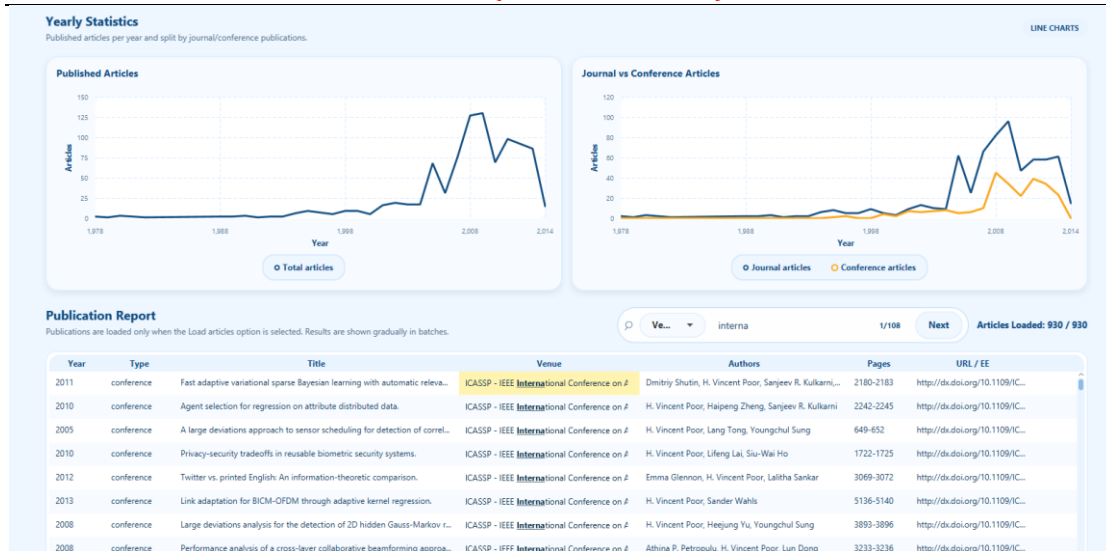
Author Profile Statistics SUMMARY

First year 1978	Last year 2014	Active years 31
Total articles 930	Journal articles 671	Conference articles 255
Distinct journals 42	Distinct conferences 29	Avg articles / year 30.00

Answer / Output:

The system displays the author profile statistics, including the first year, last year, active years, total articles, journal articles, conference articles, distinct journals, distinct conferences and average articles per year. It also presents yearly line charts for total publications and for the split between journal and conference articles. Finally, it displays a detailed publication report containing year, type, title, venue, authors, pages and URL/EE.

TEAM: (5177, 5267, 4741)

**Example 4: Venue Analysis**

Question:

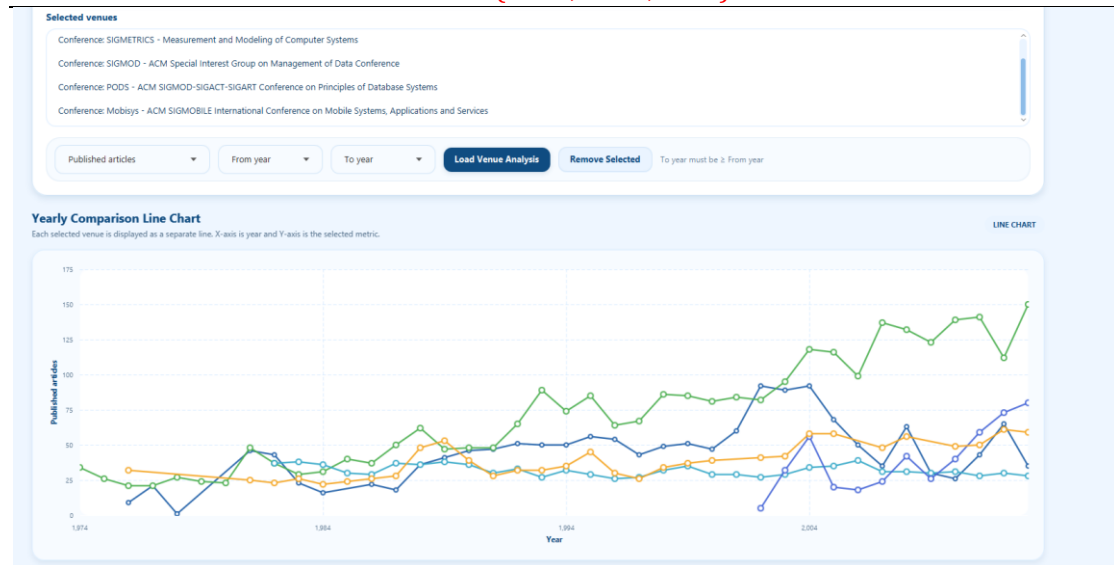
How do selected venues compare in terms of publication activity?

User actions:

The user selects several venues from the Venue Analysis page. Then, the user chooses the metric Published articles from the selection menu. Optionally, the user may define a start year and an end year. After pressing the Load Venue Analysis button, the application generates the comparison charts.

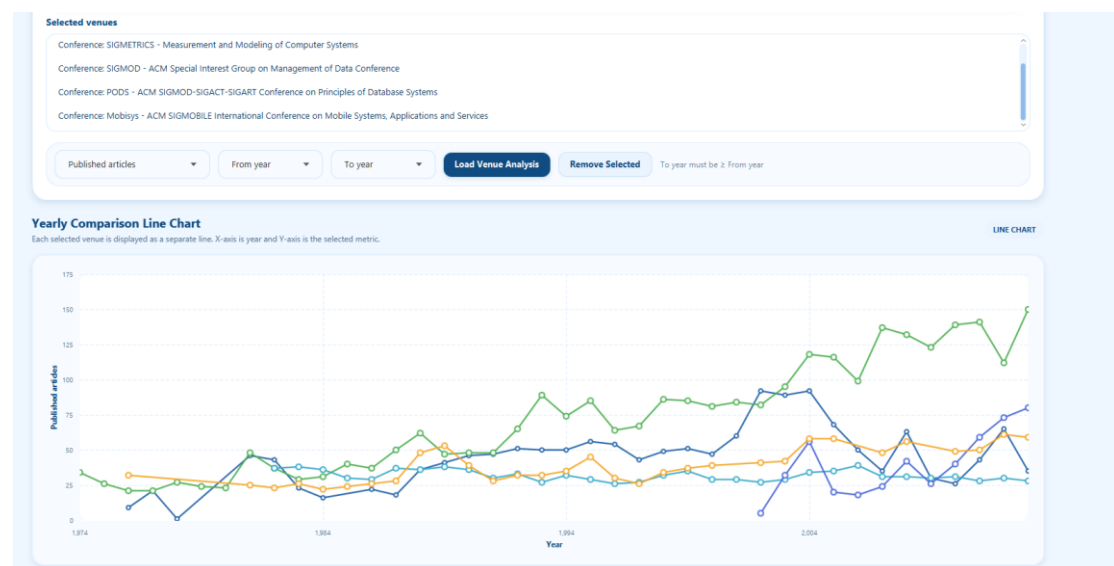


TEAM: (5177, 5267, 4741)



Answer / Output:

The application displays a yearly comparison line chart, where each selected venue is represented by a separate line and the Y-axis corresponds to the selected metric. In this example, the selected metric is Published articles, so the chart shows how the number of published articles changes over time for each venue.



The application also displays aggregate bar charts that summarize the selected venues through total articles, average articles per year and average author entries per year. This helps the user compare the overall activity and productivity of the selected journals and conferences.



Example 5: Venue Categories

Question:

How does the number of venues in a selected category change over time?

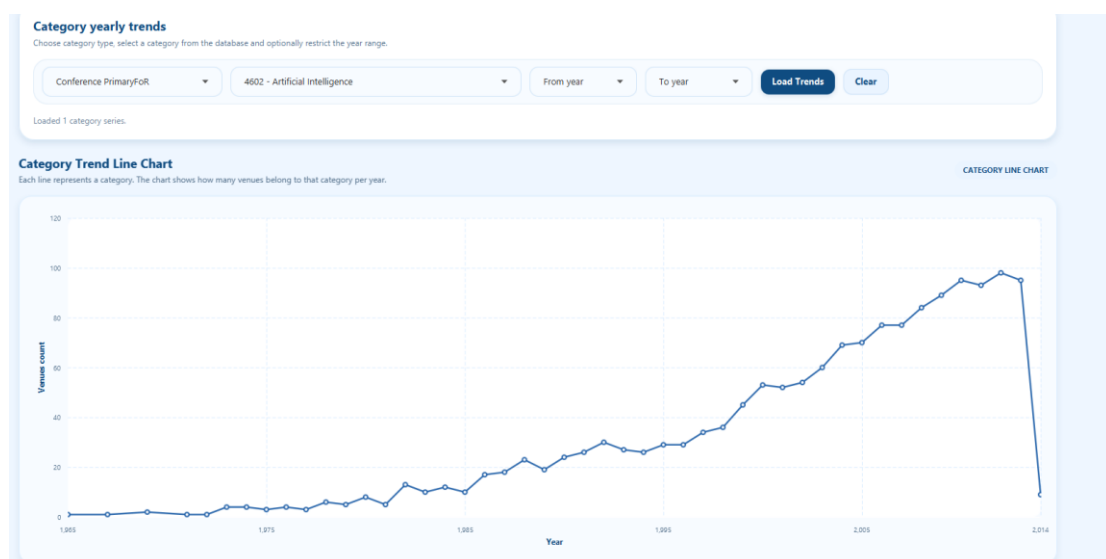
User actions:

The user opens the Visual Analytics section and selects the Venue Categories page. Then, the user chooses the category type, for example Conference PrimaryFoR or Journal Best Subject Area. After that, the user selects a specific category from the database, such as Artificial Intelligence. The user may also optionally restrict the results by selecting a start year and an end year. Finally, the user presses the Load Trends button.

Answer / Output:

The application displays a line chart showing the yearly trend of venues that belong to the selected category. Each line represents a selected category, while the X-axis represents the publication year and the Y-axis represents the number of venues.

In this example, the selected category type is Conference PrimaryFoR and the selected category is Artificial Intelligence. The chart shows how many conference venues belong to this Primary FoR category for each year, allowing the user to observe the evolution of this research area over time.



Example 6: Publisher Analysis

Question:

How do selected publishers compare in terms of total publications and journal quartile distribution?

User actions:

The user opens the Publisher Analysis page and searches for publishers. The application shows matching publishers, and next to each publisher name it displays in parentheses the total number of related articles in the dataset. The user adds several publishers to the selected list and presses Load Publisher Analysis.

Select publishers
Search a publisher, add it to the selected list, and then load the analysis. You can compare up to 10 publishers.

publ

Suggested publishers

- Old City Publishing Inc (765 publications)
- Pleiades Publishing Inc (900 publications)
- Springer/Plenum Publishers (2710 publications)
- Emerald Group Publishing Ltd (5729 publications)

Selected publishers

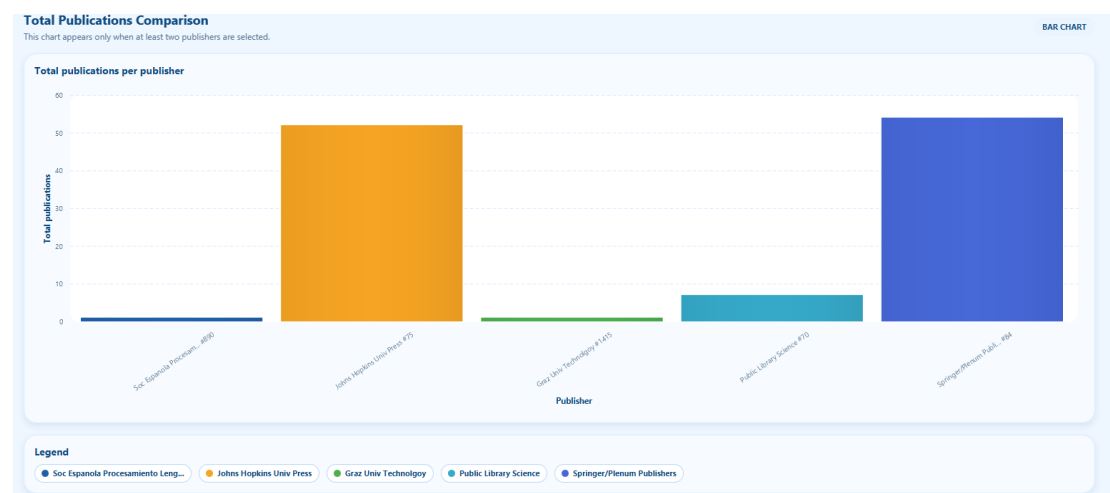
- Johns Hopkins Univ Press (975 publications)
- Graz Univ Technology (2129 publications)
- Public Library Science (2392 publications)**
- Springer/Plenum Publishers (2710 publications)

Load Publisher Analysis **Remove Selected**

Loaded analysis for 5 selected publishers.

Answer / Output:

The application first displays a bar chart with the total publications per publisher. This chart is based on the total number of articles associated with each selected publisher.



Then, the application displays a quartile distribution chart. In this chart, the analysis is based on the distinct journals of each selected publisher, grouped by quartile (Q1, Q2, Q3 and Q4). Therefore, the quartile statistics describe the journal portfolio of each publisher rather than the raw number of articles alone.

Select publishers
Search a publisher, add it to the selected list, and then load the analysis. You can compare up to 10 publishers.

publ

Suggested publishers

- Old City Publishing Inc (765 publications)
- Pleiades Publishing Inc (900 publications)
- Springer/Plenum Publishers (2710 publications)
- Emerald Group Publishing Ltd (5729 publications)

Selected publishers

- Johns Hopkins Univ Press (975 publications)
- Graz Univ Technologoy (2129 publications)
- Public Library Science (2392 publications)**
- Springer/Plenum Publishers (2710 publications)

Load Publisher Analysis Remove Selected

Loaded analysis for 5 selected publishers.

Example 7: Venue Scatter Plot

Question:

How do selected conference venues compare in terms of articles per year and average authors per article?

User actions:

The user opens the Venue Scatter Plot page, selects Conference as the venue type, searches for venues using the keyword “sigm”, and adds the desired conferences to the selected list. The user may optionally restrict the analysis to a specific year range and then presses Load Venue Scatter.

Venue Scatter Plot
Select journals or conferences. Each point represents one year for one venue.

sigm Conference Search Add

Suggested results

- SIGMETRICS - Measurement and Modeling of Computer Systems
- SIGMOD - ACM Special Interest Group on Management of Data Conference
- PODS - ACM SIGMOD-SIGACT-SIGART Conference on Principles of Database Systems
- Mobisys - ACM SIGMOBILE International Conference on Mobile Systems, Applications and Services

Selected venues for scatter plot

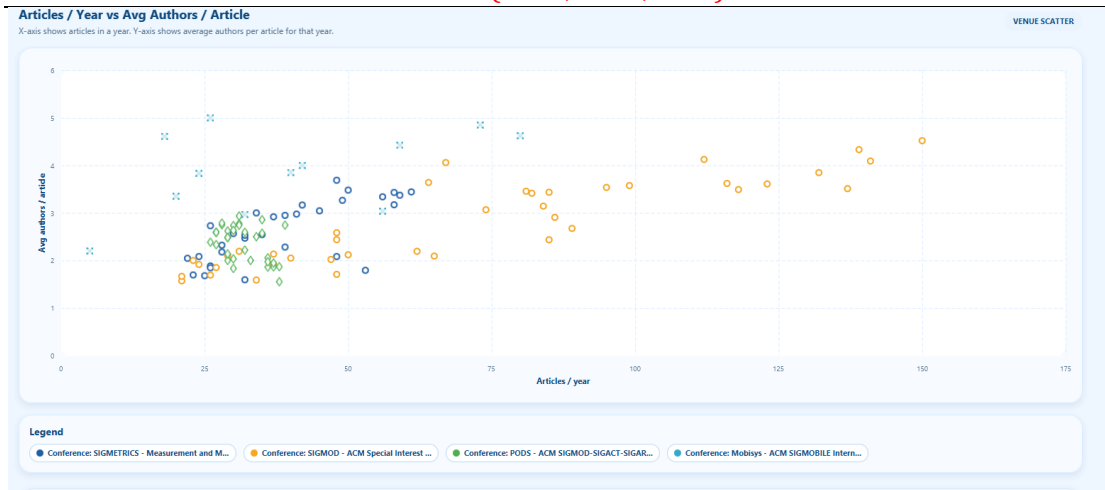
- Conference: SIGMETRICS - Measurement and Modeling of Computer Systems
- Conference: SIGMOD - ACM Special Interest Group on Management of Data Conference
- Conference: PODS - ACM SIGMOD-SIGACT-SIGART Conference on Principles of Database Systems
- Conference: Mobisys - ACM SIGMOBILE International Conference on Mobile Systems, Applications and Services

From year To year Load Venue Scatter Remove Selected Clear

Loaded 4 venue series.

Answer / Output:

The application generates a scatter plot where each point corresponds to one year for one selected conference. The X-axis represents the number of articles published in that year, and the Y-axis represents the average number of authors per article. Each selected conference is displayed as a separate series, allowing the user to compare publication volume and collaboration behavior across venues and across years.



Example 8: Journal Ranking Scatter Plot

Question:

Is there a relationship between the total number of documents of a journal and its citation impact?

User actions:

The user opens the Journal Ranking Scatter Plot page from the Visual Analytics section. The user selects Total Docs as the X-axis metric and Cites / Doc 2y as the Y-axis metric. Then, the user selects a Top N value, for example 100, and presses Load Ranking Scatter.

Answer / Output:

The application generates a scatter plot where each point corresponds to one journal. The X-axis shows the total number of documents, while the Y-axis shows the Cites / Doc 2y metric. The chart helps the user observe possible correlations between publication volume and citation impact in the journal ranking dataset.

